

Document number: P2166R1

Project: Programming Language C++

Audience: LEWG-I, LEWG, LWG

Yuriy Chernyshov <georgthegreat@gmail.com>, <thegeorg@yandex-team.ru>

Date: 2020-09-06

A Proposal to Prohibit `std::basic_string` and `std::basic_string_view` construction from `nullptr`.

Revision History

R0 -> R1

1. Include some code listings from the open source software
2. Add reference to P2037.

Introduction and Motivation

According to the C++ Standard, the behavior of `std::basic_string::basic_string(const CharT* s)` constructor *is undefined if $[s, s + \text{Traits::length}(s))$ is not a valid range (for example, if s is a null pointer)* (citation is taken [from cppreference.com](http://en.cppreference.com), the standard have slightly different wording in [21.3.2.2 \[string.cons\]](http://ericniebler.com/2014/07/21/string-cons/)). Same applies to `std::basic_string_view::basic_string_view(const CharT* s)` constructor.

Existing implementations (i. e. [libc++](http://en.cppreference.com)) might add a runtime assertion to forbid such behavior. Certain OpenSource projects would trigger this assertion. The list includes, but not limited to:

LLVM

LLVM project [had](https://github.com/llvm-project/llvm/blob/master/llvm/utils/TableGen/DFAEmitter.cpp) the following code in `llvm/utils/TableGen/DFAEmitter.cpp`:

```
struct Action {
    Record *R = nullptr;
    unsigned I = 0;
    std::string S = nullptr;

    // more code here
}
```

According to the comments, `struct Action` was intended to be an ad hoc implementation of `std::variant<Record *, unsigned, std::string>`. The bug was fixed in [D87185](https://github.com/llvm-project/llvm/pull/38718).

poco

Poco project [uses](https://github.com/pocoproject/poco/blob/master/src/Network/ODBC/ODBCProtocol.cpp) the following generic code for extracting data in their ODBC protocol implementation:

```
template<typename T>
bool extractManualImpl(std::size_t pos, T& val, SQLSMALLINT cType)
{
    SQLRETURN rc = 0;
```

```

T value = (T) 0;
resizeLengths(pos);
rc = SQLGetData(_rStmt,
    (SQLUSMALLINT) pos + 1,
    cType, //C data type
    &value, //returned value
    0, //buffer length (ignored)
    &_lengths[pos]); //length indicator

    // more code below
}

```

The project also has the `Poco::Data::LOB` class for storing Large Objects, with one of the **implicit** constructors implemented as follows:

```

LOB(const std::basic_string<T>& content):
    _pContent(new std::vector<T>(content.begin(), content.end()))
    /// Creates a LOB from a string.
{
}

```

Once `extractManualImpl` is called with an instance of `LOB`, the `nullptr` constructor would lead to undefined behavior. Such invocation could be found in `bool Extractor::extract(std::size_t pos, Poco::Data::BLOB& val)` method [in extractor.cpp](#).

protobuf

Well known Google protobuf project had the similar problem fixed in [this commit](#). The code was:

```

string GetCapitalizedType(const FieldDescriptor* field) {

    switch (field->type()) {
        // handle all possible enum values, but without adding default label
    }

    // Some compilers report reaching end of function even though all cases of
    // the enum are handed in the switch.
    GOOGLE_LOG(FATAL) << "Can't get here.";
    return NULL;
}

```

As the code is unreachable, it would not cause any problems though.

On a large private monorepo applying proposed changes and running an automatic CI-check helped to find 7 problematic projects (the number includes projects listed above), two of which would actually segfault if the code point was reached (and it was indeed reachable).

This proposal attempts to improve the diagnostics by explicitly deleting the problematic constructors, thus moving these assertions to compile time.

Impact on the Standard

This proposal changes `<string>` and `<string_view>` headers only and does not affect the language core.

Proposed Wording

The wording is relative to [N4861](#).

1. Modify [21.3.2 \[basic.string\]](#) as follows:

```
[...]
namespace std {
    template<class charT, class traits = char_traits<charT>,
            class Allocator = allocator<charT>>
    class basic_string {
    public:
        // types
        [...]
        // [string.cons], construct/copy/destroy
        [...]
        constexpr basic_string(const charT* s, size_type n, const Allocator& a = Allocator());
        constexpr basic_string(const charT* s, const Allocator& a = Allocator());
+       constexpr basic_string(nullptr_t) = delete;
        [...]
        template<class T>
        constexpr basic_string& operator=(const T& t);
        constexpr basic_string& operator=(const charT* s);
+       constexpr basic_string& operator=(nullptr_t) = delete;
        [...]
    };
    [...]
}
```

2. Modify [21.4.1 \[string.view.synop\]](#) as indicated:

```
[...]
template<class charT, class traits = char_traits<charT>>
class basic_string_view {
public:
    // types
    [...]
    constexpr basic_string_view(const charT* str);
+   constexpr basic_string_view(nullptr_t) = delete;
    [...]
};
```

Further Discourse

These changes would not allow to remove runtime check, the following code will remain compilable and will trigger the assertion:

```
const char *p = nullptr; // or more likely, p = functionThatCanReturnNull()
string s(p, 3);
```

As a development of the above proposal it seems logical to remove sized counterpart of nullptr constructors, as *the behavior is undefined if $[s, s + \text{count})$ is not a valid range* (citation source is the same). That is, the following statements are suggested where appropriate:

```
basic_string(nullptr_t, size_t) == delete;
constexpr basic_string_view(nullptr_t, size_t) == delete;
```

These changes will break the legal, yet not legitimate case of constructing `std::string` using `basic_string(nullptr, 0)`; and `std::string_view` using `basic_string_view(nullptr, 0)`; and thus they were not included into the main text of the proposal.

Acknowledgements

The author would like to thank Antony Poloukhin, Marshall Clow and Eric Fiselier for a thorough review and suggestions.

Similar problem with assignment operator is being solved by Andrzej Krzemiński in [P2037](#).